

Jackfruit Phase-1
Software Requirements Specification
for
SnakeXtreme – Multi-Mode Snake Game with Power-ups & Leaderboard

Field	Details
Version	1.0 — Initial (Requirements Stage)
Prepared by	Team 16 — Ayushi Srivastava, B V Tejas, Bharath Hegde, C Dinesh Kumar
Organization	PES University, Dept. of CSE — UE24CS341A Software Engineering
Date Created	31 August 2026

Revision History

Name	Date	Reason for Changes	Version
Team 16	31-Aug-2026	Initial SRS draft for Requirements submission	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of SnakeXtreme, a desktop, single-player Snake game developed in C++ using the SFML library. It covers functional, non-functional and security requirements for the complete system: core gameplay, game modes and power-ups, scoring and leaderboard persistence, and menu/settings navigation. This SRS is the baseline against which the Design, Implementation and Test Case deliverables of the project will be validated.

1.2 Intended Audience and Reading Suggestions

This document is intended for: the four development-team members (as designers, implementers and testers of their owned features), the course instructor/evaluator, and any peer reviewer of the project. Section 2 gives an overall description of the product; Sections 3–4 describe interfaces and analysis models; Section 5 lists detailed functional requirements grouped by feature/owner; Sections 6–7 cover non-functional, security and other requirements; the Appendices provide a glossary, file field layouts and the requirement traceability matrix.

1.3 Product Scope

SnakeXtreme digitizes and extends the classic Snake arcade concept into a replayable desktop game with three selectable game modes, three difficulty levels, three power-up types, mode-specific obstacles, a persistent local Top-10 leaderboard, and a settings system for controls/audio/theme. The product benefits the course's Agile/SE learning objectives by decomposing cleanly into four independently ownable, testable functional modules while remaining a self-contained, non-networked application with no external customer dependency.

1.4 References

- PES University — Jackfruit Mini-Project: Software Engineering Project Guidelines 2026 (UE24CS341A).
- PES University — Software Engineering Level 3 Student Submission Guide (Coffee Kiosk Exemplar).
- SFML 2.5 Official Documentation — <https://www.sfml-dev.org/documentation/2.5.1/>
- Team 16 Synopsis / Project Proposal, Version 1.0.

2. Overall Description

2.1 Product Perspective

SnakeXtreme is a new, self-contained desktop product; it is not a follow-on to an existing system and has no external system dependencies beyond the local operating system, filesystem and the SFML runtime libraries. The system consists of a single executable comprising a Presentation/Rendering layer (SFML window, sprites, HUD), a Game Logic layer (movement, collision, modes, power-ups, scoring), and a Persistence layer (leaderboard and settings files on local disk).

2.2 Product Functions

At a high level the product lets the player:

- Navigate a main menu to start a new game, view the leaderboard, adjust settings, or exit.
- Select a game mode (Classic / Timed Challenge / Obstacle) and a difficulty level (Easy / Medium / Hard).
- Control a snake on a grid to consume food, grow, and avoid ending the game via wall, self, or obstacle collisions.
- Collect time-limited power-ups (Speed Boost, Score Multiplier, Shrink) that temporarily modify gameplay.

- Pause and resume an active game, and view a game-over summary with the final score.
- Have qualifying scores persisted to a local Top-10 leaderboard, viewable from the main menu.
- Customize and persist control key bindings, audio volume, and visual theme.

2.3 Scope

The table below fixes the boundary of Version 1.0 so that requirements, design and testing are not written against an open-ended feature set.

In Scope	Out of Scope
Single-player, single-machine gameplay	Multiplayer / networked gameplay
Classic, Timed Challenge and Obstacle game modes	Mobile or web-based versions
Easy / Medium / Hard difficulty levels	Cloud-hosted or online leaderboard
Speed, Multiplier and Shrink power-ups	AI-controlled opponent snake
Local file-based leaderboard (Top 10) and settings persistence	In-app purchases / monetization
Keyboard-based controls, sound effects and background music	Procedurally generated levels beyond obstacle placement

2.4 User Classes and Characteristics

SnakeXtreme has a single human-facing role. The Local File System referenced elsewhere in this document (Section 3.2, Appendix B) is a persistence dependency, not a user class, and is therefore modelled only as a secondary actor in the use-case diagram, not below.

User Class	Privileges	Characteristics
Player	Start/pause/resume games; select mode and difficulty; view leaderboard; change settings	Non-technical, casual desktop user. Includes both first-time players (need an immediately learnable menu flow) and returning players (primarily interested in beating their leaderboard rank). No privilege tiers — every player has identical access to all features.

2.5 Operating Environment

The application targets Windows 10/11 (primary) desktop environments with a minimum screen resolution of 800×600, running as a native compiled C++17 executable linked against SFML 2.5+. Linux compatibility (via SFML's cross-platform support) is a stretch goal and not a Version 1.0 requirement.

2.6 Design and Implementation Constraints

- Must be implemented in C++17 using the SFML library, as assigned for this project.
- Game logic must be separated from the SFML rendering layer to remain unit-testable without a display context.
- No external network calls or backend server — all persistence is local-file-based.
- Must follow Agile methodology with sprint-wise, traceable Git/GitHub activity from every team member.
- Grid size fixed at 30×20 cells for Version 1.0 to keep collision and rendering logic consistent across modes.

2.7 Assumptions and Dependencies

- Assumes the target machine has a graphics driver compatible with SFML's OpenGL-based rendering.
- Assumes a keyboard is available; no touch or gamepad support is planned for Version 1.0.
- Depends on the SFML 2.5+ runtime and the nlohmann/json library (or equivalent) for persistence.
- Assumes single-user, single-instance execution — concurrent multi-instance leaderboard writes are out of scope.

3. External Interface Requirements

3.1 User Interfaces

The game presents a 2D SFML-rendered window (default 900×600 px) with the following screens: Main Menu, Mode/Difficulty Selection, Gameplay (with HUD showing score, mode, timer where applicable), Pause overlay, Game Over screen, Leaderboard screen, and Settings screen. All screens share a consistent navigation convention: Enter/Click to confirm, Esc to go back or pause, and arrow keys/WASD for menu and gameplay navigation. Error/validation messages (e.g., invalid name entry) are displayed as inline text below the relevant field.

3.2 Software Interfaces

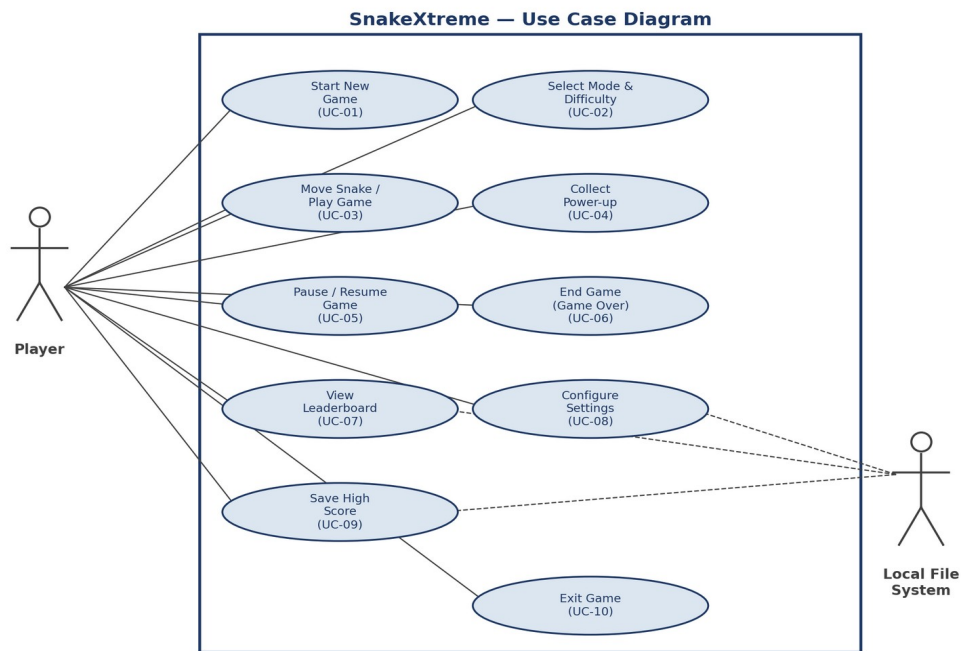
Component	Version	Purpose
SFML — Graphics module	2.5+	Window creation, 2D rendering of grid, snake, food, power-ups, UI
SFML — Audio module	2.5+	Background music and sound-effect playback
SFML — Window / System modules	2.5+	Event loop, keyboard input, timing/clock
nlohmann/json (or equivalent)	3.x	Serializing/deserializing leaderboard.json and config.json
Local Filesystem	OS-native	Persistent storage of leaderboard and settings files

3.3 Communications Interfaces

None. SnakeXtreme is a fully offline, single-machine application for Version 1.0. No network, email, or web-service communication is required or supported.

4. Analysis Models

The primary actor is the Player; the Local File System is modelled as a secondary actor representing persistence. The use case diagram below shows all major use cases identified for Version 1.0.



Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Start New Game	Player	Application launched	Player name captured; navigates to mode selection
UC-02	Select Game Mode & Difficulty	Player	UC-01 complete	Mode and difficulty set for the session
UC-03	Move Snake / Play Game	Player	Mode/difficulty selected	Snake moves, grows, and score updates in real time
UC-04	Collect Power-up	Player	Power-up present on grid	Temporary gameplay effect applied
UC-05	Pause / Resume Game	Player	Game in progress	Game loop suspended/resumed
UC-06	End Game (Game Over)	Player	Collision or timer expiry occurs	Game-over screen shown with final score
UC-07	View Leaderboard	Player	Leaderboard file exists (or defaults to empty)	Top-10 scores displayed
UC-08	Configure Settings	Player	Settings menu opened	Controls/audio/theme updated and saved
UC-09	Save High Score	Player / File System	Game over and score qualifies	Leaderboard file updated with new entry
UC-10	Exit Game	Player	Any screen active	Application terminates; settings saved

5. System Features

Functional requirements are organized below by system feature, aligned to the four owned functionalities defined in the Synopsis. Each requirement is uniquely tagged and is written to be concise, complete, unambiguous, verifiable and necessary — vague, unmeasurable phrasing (e.g. "the snake should move smoothly") is deliberately avoided in favor of concrete, testable values wherever a requirement could otherwise be interpreted subjectively.

5.1 Feature 1 — Core Gameplay Engine (Owner: Ayushi Srivastava)

Description & Priority: High. Provides the foundational grid, movement, collision and food logic on which every other feature depends.

Stimulus/Response: Player presses a direction key → snake's heading updates (unless it reverses onto itself) → on each game-loop tick the snake advances one cell → collisions and food consumption are evaluated.

ID	Requirement	Priority	Verification
FR-001	The system shall render the game grid and snake within 2 seconds of launching a new game.	High	Functional test
FR-002	The system shall advance the snake by one grid cell at a fixed tick interval per difficulty: Easy = 200 ms/cell, Medium = 120 ms/cell, Hard = 70 ms/cell.	High	Functional + timing test
FR-003	The system shall change the snake's direction based on arrow-key/WASD input, except a 180° reversal onto its own body.	High	Functional + negative test
FR-004	The system shall end the current game when the snake's head collides with a boundary wall.	High	Functional test
FR-005	The system shall end the current game when the snake's head collides with any segment of its own body.	High	Functional test
FR-006	The system shall spawn a new food item at a random unoccupied grid cell immediately after the previous food is consumed.	High	Functional test
FR-007	The system shall increase the snake's length by exactly one segment when the snake's head occupies the same cell as food.	High	Unit test

5.2 Feature 2 — Game Modes, Difficulty & Power-up/Obstacle System (Owner: B V Tejas)

Description & Priority: High. Extends the core engine with selectable play styles, scaling challenge, and reward/hazard variety.

Stimulus/Response: Player selects a mode/difficulty at UC-02 → system configures timer/obstacles/speed accordingly → during play, power-ups spawn periodically and apply timed effects when collected.

ID	Requirement	Priority	Verification
FR-008	The system shall allow the player to select one of three game modes — Classic, Timed Challenge, or Obstacle — before starting a game.	High	Functional test
FR-009	The system shall allow the player to select a difficulty level (Easy, Medium, Hard) that determines snake movement speed.	High	Functional test
FR-010	In Timed Challenge mode, the system shall end the game when the countdown timer reaches zero.	High	Functional test
FR-011	In Obstacle mode, the system shall place static obstacles on the grid that end the game upon collision with the snake's head.	High	Functional test
FR-012	The system shall spawn a power-up item at a random unoccupied	Medium	Functional test

ID	Requirement	Priority	Verification
	cell at intervals not exceeding 15 seconds during active gameplay.		
FR-013	The system shall apply a temporary speed-boost effect for 5 seconds when the snake collects a Speed power-up.	Medium	Unit + functional test
FR-014	The system shall apply a ×2 score-multiplier effect for 10 seconds when the snake collects a Multiplier power-up.	Medium	Unit test
FR-015	The system shall reduce the snake's length by one segment (minimum length 3) when the snake collects a Shrink power-up.	Medium	Unit + boundary test

5.3 Feature 3 — Scoring, Leaderboard & Persistence (Owner: Bharath Hegde)

Description & Priority: High. Owns score calculation and all local persistence, ensuring player progress survives across sessions.

Stimulus/Response: Food/power-up consumption → score recalculated and redrawn on HUD; Game over with a qualifying score → leaderboard file updated; Player opens Leaderboard → file loaded and displayed.

ID	Requirement	Priority	Verification
FR-016	The system shall increment the player's score by a mode-specific value each time food is consumed.	High	Unit test
FR-017	The system shall display the current score on-screen in real time during gameplay.	High	Functional test
FR-018	The system shall persist the Top-10 high scores (player name, score, mode, difficulty, date) to a local leaderboard file.	High	Integration test
FR-019	The system shall load and display the persisted leaderboard when the player selects 'View Leaderboard' from the main menu.	High	Functional test
FR-020	The system shall insert a new score into the leaderboard file only if it ranks within the current top 10.	High	Unit + boundary test
FR-021	The system shall validate leaderboard file structure on load and revert to a default empty leaderboard if validation fails.	Medium	Negative test

5.4 Feature 4 — Menu, UI Navigation & Player Settings (Owner: C Dinesh Kumar)

Description & Priority: High. Owns every screen transition outside active gameplay and all player-configurable preferences.

Stimulus/Response: Player launches app → main menu displayed; Player selects a menu option → corresponding screen loads; Player changes a setting → preference applied immediately and persisted.

ID	Requirement	Priority	Verification
FR-022	The system shall display a main menu with options New Game, Leaderboard, Settings, and Exit on application launch.	High	Functional test
FR-023	The system shall prompt the player to enter a display name (maximum 15 characters) before starting a new game.	High	Functional + boundary test
FR-024	The system shall allow the player to pause and resume an active game using the Esc key.	High	Functional test
FR-025	The system shall display a Game Over screen showing the final score with options to restart or return to the main menu.	High	Functional test
FR-026	The system shall allow the player to remap the four movement control keys via the Settings menu.	Medium	Functional test
FR-027	The system shall allow the player to independently adjust	Medium	Functional test

ID	Requirement	Priority	Verification
	background-music and sound-effect volume via the Settings menu.		
FR-028	The system shall allow the player to select a visual theme/skin for the snake and grid via the Settings menu.	Low	Functional test
FR-029	The system shall persist player settings (controls, volume, theme) across application restarts using a local configuration file.	High	Integration test

6. Other Non-Functional Requirements

6.1 Performance Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	The game loop shall maintain a minimum of 30 frames per second on the target hardware profile.	Performance test
NFR-002	Performance	Input-to-response latency for a direction-key change shall not exceed 100 ms.	Performance test
NFR-003	Reliability	The application shall not crash when the leaderboard or config file is missing/corrupted; it shall fall back to defaults.	Negative/ integration test
NFR-004	Usability	A new player shall be able to start their first game within 3 primary screens of launching the application.	Usability walkthrough
NFR-005	Maintainability	Game logic shall be separated into independently identifiable modules (engine, modes, persistence, UI) with unit tests.	Code review
NFR-006	Portability	The application shall build and run on Windows 10/11 using SFML 2.5+; Linux support is a stretch objective.	Build verification
NFR-007	Testability	Core gameplay logic shall be exposed via functions/classes that are testable without an active SFML rendering context.	Unit test

6.2 Safety Requirements

SnakeXtreme is a non-safety-critical desktop game; no physical or health-related safety hazards are applicable. The only safety-adjacent concern is data safety: the application must never leave the leaderboard or settings file in a corrupted, unreadable state after an unexpected termination (see SEC-004).

6.3 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Player name input shall be sanitized to reject delimiter/control characters before being written to the leaderboard file, to prevent file-format injection.	Negative/security test
SEC-002	The application shall not collect, require, or store any personally identifiable information beyond the player-entered display name.	Design review
SEC-003	Configuration and leaderboard files shall be validated against their expected schema before being parsed, to prevent malformed-file crashes.	Negative/security test
SEC-004	File write operations for leaderboard and settings data shall use a safe write pattern (write-to-temp-then-rename) to avoid data corruption on unexpected termination.	Integration/fault-injection test
SEC-005	The system shall treat all data loaded from leaderboard/config files as untrusted and shall parse it only as data, never execute or interpret it as code.	Security review
SEC-006	The system shall enforce a fixed maximum buffer size for the player-name input field to prevent buffer-overflow conditions in the underlying C++ implementation.	Boundary/security test

6.4 Software Quality Attributes

- Maintainability is prioritized over raw performance: the engine/mode/persistence/UI split should allow one module to change with minimal impact on others.
- Usability favors ease-of-learning: a first-time player should understand controls without reading external instructions.
- Reliability is prioritized for persistence: leaderboard/settings corruption must never crash the application.

6.5 Business Rules / Domain Rules

- A game session belongs to exactly one game mode and one difficulty level for its entire duration.
- Only scores from a completed game (i.e., a game-over event has occurred) are eligible for leaderboard submission.
- The snake's minimum length is 3 segments; the Shrink power-up cannot reduce it below this value.
- Obstacles are only present and only end the game in Obstacle mode.
- Only one power-up effect may be active on the snake at a time; a newly collected power-up replaces any active effect.

7. Other Requirements

- Data retention: leaderboard entries persist indefinitely on local disk until manually cleared by the player (no automatic expiry in Version 1.0).
- Localization: Version 1.0 supports English text only; no internationalization is planned.
- Reuse: SFML, and a JSON serialization library, are reused third-party components; all gameplay/persistence logic is developed by the team.
- Legal: No third-party copyrighted art/audio assets will be used without an appropriate open license; original or royalty-free assets only.

7.1 Initial Validation Specification (Planning Only)

At this Requirements stage, validation planning identifies how each requirement category will eventually be validated; execution and results are recorded later in the Implementation & Testing and System Validation deliverables.

Requirement Category	Planned Validation	Example
Functional (Core Engine)	Black-box functional tests + unit tests on movement/collision logic	Snake reverses direction only when no opposite-key input follows immediately
Functional (Modes/Power-ups)	Black-box functional tests per mode; mock-based unit tests for power-up timers	Timed mode ends exactly at 0:00; Speed boost reverts after 5s
Persistence	Integration tests with valid/corrupted/missing files	Leaderboard loads correctly; corrupted file falls back to defaults
Security	Negative/security tests on file schema and name-input sanitization	Player enters a name containing a delimiter character; input is rejected/sanitized
Performance	Frame-rate and input-latency measurement under normal load	Game loop sustains ≥ 30 FPS over a 5-minute play session
Usability	Task-based walkthrough with a first-time player	New player reaches first gameplay screen within 3 menu screens

Appendix A: Glossary

Term	Definition
SFML	Simple and Fast Multimedia Library — a C++ library for 2D graphics, audio, window and input handling.
Grid	The fixed 30×20 cell playing field on which the snake, food, power-ups and obstacles are positioned.
Game Mode	One of Classic, Timed Challenge, or Obstacle — determines win/lose conditions and hazards present.
Difficulty	Easy, Medium, or Hard — determines the snake's base movement speed.
Power-up	A collectible item (Speed, Multiplier, Shrink) that applies a temporary effect to the snake or score.
Obstacle	A static grid cell that ends the game on collision, present only in Obstacle mode.
Leaderboard	A locally persisted, ranked list of the Top-10 highest scores achieved across sessions.
HUD	Heads-Up Display — the on-screen overlay showing score, mode, and (where applicable) remaining time.
Config File	A local file storing the player's persisted settings (controls, volume, theme).

Appendix B: Field Layouts

Field layout for a single leaderboard entry, persisted in leaderboard.json:

Field	Length	Data Type	Description	Mandatory
player_name	15	String	Display name entered by the player before the game	Y
score	10	Integer	Final score achieved in the game	Y
game_mode	20	String (enum)	One of: Classic, Timed, Obstacle	Y
difficulty	10	String (enum)	One of: Easy, Medium, Hard	Y
date_achieved	10	Date (YYYY-MM-DD)	Date the score was achieved	Y

Field layout for player settings, persisted in config.json:

Field	Length	Data Type	Description	Mandatory
key_up / key_down / key_left / key_right	1	Key code	Remapped movement control keys	Y
music_volume	3	Integer (0-100)	Background music volume level	Y
sfx_volume	3	Integer (0-100)	Sound-effect volume level	Y
theme	10	String (enum)	One of: Classic, Neon, Retro	Y

Appendix C: Requirement Traceability Matrix (Initial)

At the Requirements stage, only the Requirement ID and description are finalized; Architecture, Design, Code and Test references are populated progressively in the Design and Implementation & Testing deliverables and are marked TBD here.

Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID
FR-001-007	Core gameplay engine (movement, collision, food)	TBD	TBD	TBD	TBD	TBD
FR-008-015	Game modes, difficulty & power-ups/obstacles	TBD	TBD	TBD	TBD	TBD
FR-016-021	Scoring & leaderboard persistence	TBD	TBD	TBD	TBD	TBD
FR-022-029	Menu navigation & player settings	TBD	TBD	TBD	TBD	TBD
NFR-001-007	Performance, reliability, usability, maintainability, portability, testability	TBD	TBD	TBD	TBD	TBD
SEC-001-006	Input sanitization, data-privacy, schema validation, safe file writes, untrusted-data handling, buffer-overflow prevention	TBD	TBD	TBD	TBD	TBD